

Assignment 1

September 30, 2026

```
[1]: import torch
import torch.nn as nn
import torchvision
from torch.utils.data import Dataset, DataLoader, TensorDataset
from torch.utils.tensorboard import SummaryWriter
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from datetime import datetime
import cv2
```

1 Part 1:

In your own words, answer the following questions. You are welcome to use, for instance, a python script to solve some of the questions, but try to get through without resorting to a chatbot.

1.1 a: In your opinion, what were the most important turning points in the history of deep learning?

Of course things like the conceptualization of the perceptron and MLP's are important, but I feel like the two most pivotal developments in the deep learning space was the LeNet architecture and the AlexNet Architecture.

LeNet was the first renowned practical use of back propagation which went on to be a foundational algorithm for deep learning. Furthermore, it led to the development of the MNIST dataset, which has become the golden standard for learning deep learning.

AlexNet was also significant since it demonstrated the power of deep learning given modern computing power, which brought the field closer to the mainstream and more importantly closer to investors. This has led to a surge in research in the field.

1.2 b: Explain the ADAM optimizer.

The ADAM optimizer is a combination a RMSProp optimizer with momentum. RMSProp is an adaptive learning rate optimization algorithm based on the Adagrad optimizer. Adagrad is modifies the stochastic gradient descent algorithm by introducing adaptive per-parameter learning rates by using the sum of previous gradients. The problem with this approach is that the learning rate can become so small that the model stops learning. This is fixed by RMSProp by introducing a decay

factor to the sum. Finally, ADAM adds momentum to RMSProp, which is a way to make the gradient descent “remember” previous gradients and move in a more smoothed out path.

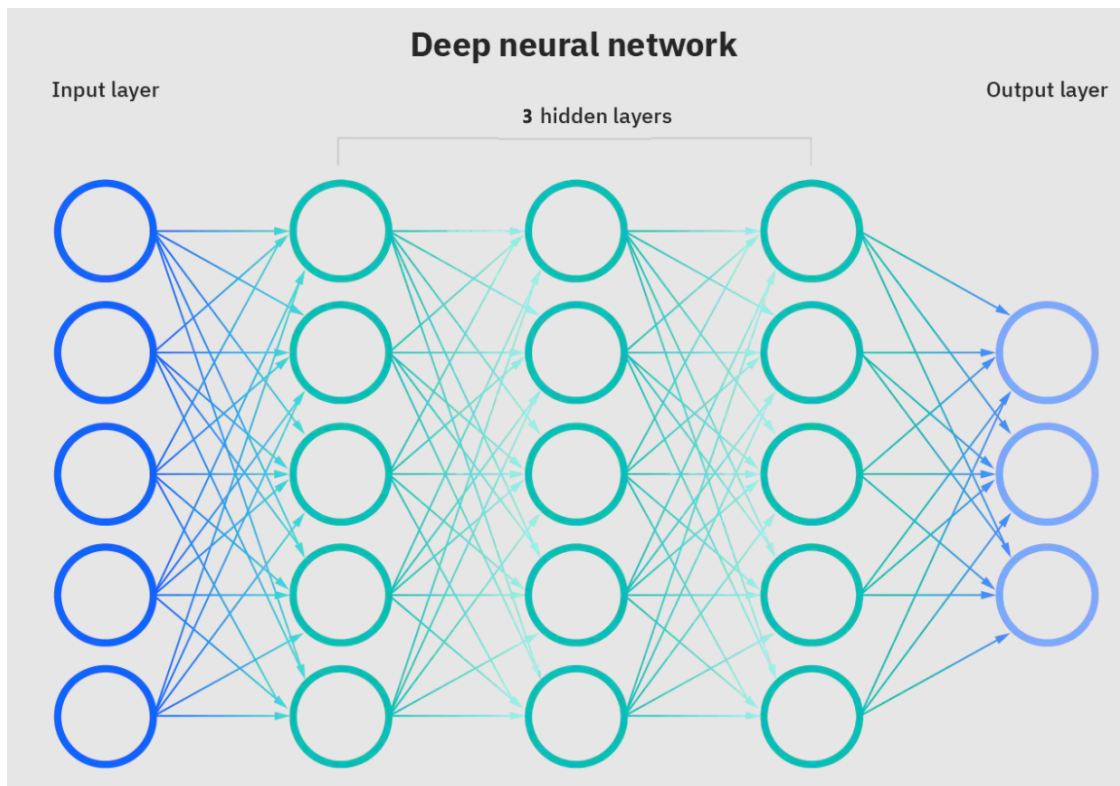
1.3 c: Assume data input is a single 30x40 pixel image. First layer is a convolutional layer with 5 filters, with kernel size 3x2, step size (1,1) and padding='valid'. What are the output dimensions?

```
[2]: img = torch.rand(1, 1, 30, 40)
conv_layer = nn.Conv2d(in_channels=1, out_channels=1, kernel_size=(3,2),
    ↪stride=(1,1), padding='valid')
output = conv_layer(img)
for i in range(4):
    output = conv_layer(output)
output.shape
```

```
[2]: torch.Size([1, 1, 20, 35])
```

The output dimensions of that operation would be 20x35.

1.4 d: Assuming ReLU activations and offsets, and that the last layer is softmax, how many parameters does this network have:



It seems that there are no biases in the images and there are no mentions of biases, so I'll assume that there are only weights.

In that case, there are 5 inputs, making it 5 parameters for each neuron. Given 5 neurons in each hidden layer with 3 hidden layers, that would make $5 \cdot 5 \cdot 3 = 75$ parameters in the hidden layers. Lastly, the hidden layer goes from 5 neurons to 3 output neurons, which is an additional $5 \cdot 3 = 15$ parameters. ReLU and softmax do not add any parameters, so in total that makes $75 + 15 = 90$ parameters.

1.5 e: For a given minibatch, the targets are [1,4, 5, 8] and the network output is [0.1,4.4,0.2,10]. If the loss function is “torch.nn.HuberLoss(reduction=‘mean’, delta=1.0)”, what is the loss for this minibatch?

```
[3]: target = torch.tensor([1, 4, 5, 8])
      output = torch.tensor([0.1, 4.4, 0.2, 10])
      loss_func = nn.HuberLoss(reduction='mean', delta=1.0)
      loss_func(output, target)
```

```
[3]: tensor(1.5713)
```

The loss of the minibatch is 1.5713

2 Part 2: Writing a PyTorch dataset

The Insects.zip-file contains images of three species of bees. The Insects.csv -file has three columns of which you should focus on the one named “species” and the one named “filename”. The “species” column tells the target label of the images listed in the “filename” column.

Your task is to write a PyTorch dataset, which provides image samples together with their target labels.

I.e.,

<image data from 1_003311_1_2020_05_09-14-21-01-479.jpg>, *Andrena fulva*

<image data from 1_206320_1_2020_03_04-11-17-03-984.jpg>, *Panurgus banksianus*

<image data from 2_208265_1_2020_03_07-10-34-31-425.jpg>, *Lasioglossum punctatissimum*

```
[4]: insects_data = pd.read_csv("insects.csv")
      counter = {}
      for filename in insects_data["filename"]:
          key = str(list(torchvision.io.read_image(f"Insects/{filename}").shape))
          counter[key] = counter.get(key, 0) + 1

      highest_value = 0
      for key, value in counter.items():
          if value > highest_value:
              highest_value = value
              highest_key = key
```

```
print(f"The most common image shape is {highest_key} with {highest_value}␣  
↪occurrences. I will therefore resize images to this shape.")
```

The most common image shape is [3, 520, 520] with 285 occurrences. I will therefore resize images to this shape.

```
[5]: class CustomInterface(Dataset):  
    def __init__(self, csv_path: str):  
        self.data = pd.read_csv(csv_path)  
        self.class_map = {  
            "Andrena fulva": 0,  
            "Panurgus banksianus": 1,  
            "Lasioglossum punctatissimum": 2  
        }  
        self.img_dim = (520, 520)  
  
    def __len__(self):  
        return len(self.data)  
  
    def __getitem__(self, idx: int):  
        row = self.data.iloc[idx]  
        img = cv2.imread(f"Insects/{row.iloc[-1]}")  
        img = cv2.resize(img, self.img_dim)  
        class_id = self.class_map[row.iloc[1]]  
        img_tensor = torch.from_numpy(img)  
        img_tensor = img_tensor.permute(2, 0, 1)  
        class_id = torch.tensor([class_id])  
        return img_tensor, class_id
```

I have changed a bit of the code in the tester script to make it run on my machine, but it is only minor changes such as adding a `if __name__ == "__main__"` block.

3 Part 3:

We supply two datasets, with points drawn from a 2D feature space, and a label assigned to each data point. The data has already been assigned to train and test sets. Each file is a table, where first column is the label, and the 2nd and 3rd columns are the features.

This exercise does not require a GPU.

3.1 a: describe & visualize the data

The train and test data have been directly copied into corresponding txt files used in the code block below.

```
[6]: test_data = {  
    "label": np.array([], dtype=int),  
    "x": np.array([], dtype=float),  
    "y": np.array([], dtype=float)
```

```

}
for line in open("testData.txt"):
    data = line.strip("\n").split(" ")
    test_data["label"] = np.append(test_data["label"], int(float(data[0])))
    test_data["x"] = np.append(test_data["x"], float(data[1]))
    test_data["y"] = np.append(test_data["y"], float(data[2]))

train_data = {
    "label": np.array([], dtype=int),
    "x": np.array([], dtype=float),
    "y": np.array([], dtype=float)
}
for line in open("trainData.txt"):
    data = line.strip("\n").split(" ")
    train_data["label"] = np.append(train_data["label"], int(float(data[0])))
    train_data["x"] = np.append(train_data["x"], float(data[1]))
    train_data["y"] = np.append(train_data["y"], float(data[2]))

```

```

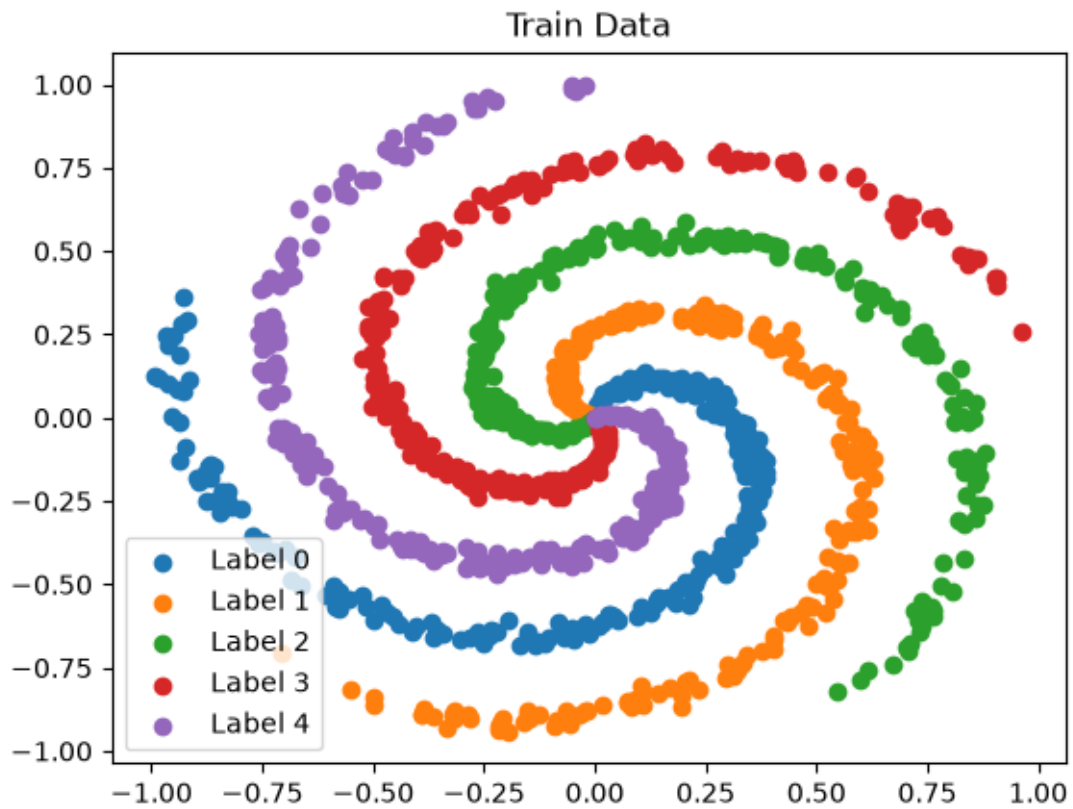
[7]: for label in np.unique(train_data["label"]):
    plt.scatter(train_data["x"][train_data["label"] == label],
        ↪train_data["y"][train_data["label"] == label], label=f"Label {int(label)}")
plt.title("Train Data")
plt.legend()

```

```

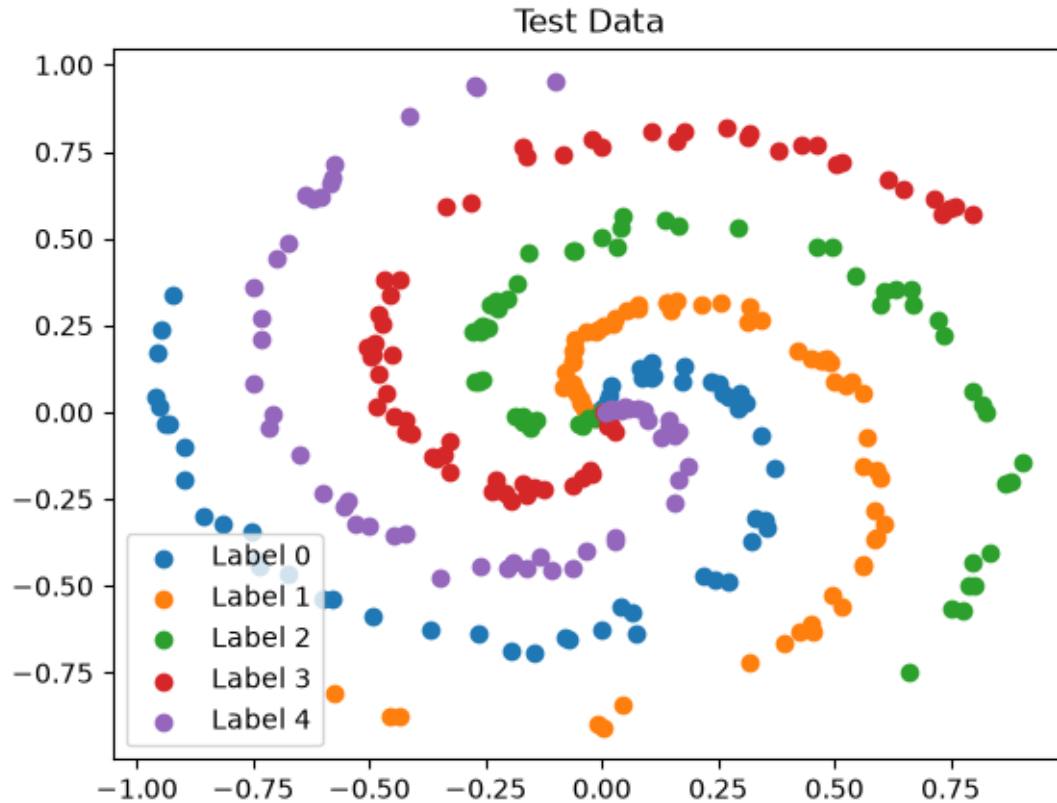
[7]: <matplotlib.legend.Legend at 0x28faf7c5160>

```



```
[8]: for label in np.unique(test_data["label"]):
      plt.scatter(test_data["x"][test_data["label"] == label],
                  test_data["y"][test_data["label"] == label], label=f"Label {int(label)}")
plt.title("Test Data")
plt.legend()
```

[8]: <matplotlib.legend.Legend at 0x28fb1a18ad0>



The data seems to be a spiral where each class represents a different arm in the spiral.

3.2 b: design a neural network using pytorch to correctly assign labels

For this exercise i took heavy inspiration from [the introduction to training in the pytorch documentation](#).

3.2.1 Describe your network

Since the problem is to define a nonlinear decision boundary between 5 classes, a single-hidden layer universal approximator with nonlinear activation functions should in theory be enough. However, to be sure the network has enough capacity 2 hidden layers will be used, which is also supported by [this article](https://www.heatonresearch.com/2017/06/01/hidden-layers.html) (<https://www.heatonresearch.com/2017/06/01/hidden-layers.html>). The input layer will have 2 neurons for each of the features in 2D space, while the output layer will have 5 neurons corresponding to the 5 labels. The hidden layers are set to have 10 neurons each. The chosen activation functions will be ReLu for both hidden layers and no activation function for the output layer since the loss function later on will be cross-entropy [which expects raw logits in pytorch](#).

```
[9]: class SpiralModel(nn.Module):
      def __init__(self):
          super(SpiralModel, self).__init__()
          self.hidden1 = nn.Linear(2, 10)
```

```

self.hidden2 = nn.Linear(10, 10)
self.output = nn.Linear(10, 5)
self.relu = nn.ReLU()

def forward(self, x):
    x = self.hidden1(x)
    x = self.relu(x)
    x = self.hidden2(x)
    x = self.relu(x)
    x = self.output(x)
    return x

model = SpiralModel()

```

3.2.2 Describe your training strategy

Since this is a classification problem, the loss function will be cross-entropy loss. To update the network weights, the ADAM optimizer will be used with an initial learning rate of 0.001. The training could be split into a validation set as well but since the data is already split into a training and testing set and the task is pretty simple, they will be used as is. The data will be divided into minibatches of size 32 and during training the data will be shuffled between epochs. All of this will run for 1000 epochs with comments during training to track performance and assess convergence.

```

[10]: training_set = TensorDataset(torch.tensor(np.column_stack((train_data["x"],
    ↪train_data["y"]))), dtype=torch.float32), torch.tensor(train_data["label"],
    ↪dtype=torch.long))
testing_set = TensorDataset(torch.tensor(np.column_stack((test_data["x"],
    ↪test_data["y"]))), dtype=torch.float32), torch.tensor(test_data["label"],
    ↪dtype=torch.long))
training_loader = DataLoader(training_set, batch_size=32, shuffle=True)
testing_loader = DataLoader(testing_set, batch_size=32, shuffle=False)
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

```

```

[11]: def train_one_epoch(loader):
    running_loss = 0.

    for inputs, labels in loader:
        optimizer.zero_grad()
        outputs = model(inputs)
        loss = loss_fn(outputs, labels)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()

    return running_loss / len(loader)

```

```

[12]: timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')
writer = SummaryWriter(f'runs/spiral_trainer_{timestamp}')
EPOCHS = 1000
best_test_loss = float('inf')

for epoch in range(EPOCHS):

    model.train()
    avg_loss = train_one_epoch(training_loader)

    running_test_loss = 0.0
    model.eval()
    correct = 0
    total = 0

    with torch.no_grad():
        for inputs, labels in testing_loader:
            outputs = model(inputs)
            loss = loss_fn(outputs, labels)
            running_test_loss += loss.item()
            _, predicted = torch.max(outputs, 1)

            total += labels.size(0)
            correct += (predicted == labels).sum().item()

    avg_test_loss = running_test_loss / len(testing_loader)
    test_accuracy = correct / total

    if epoch % 100 == 99:
        print(f'EPOCH {epoch + 1}:')
        print(
            f'LOSS train {avg_loss:.4f} '
            f'test {avg_test_loss:.4f} '
        )
        print(f'Test accuracy: {test_accuracy:.4f} ({correct} / {total} test_
↪samples).')
        print()

    writer.add_scalars(
        'Training vs. Test Loss',
        {
            'Training': avg_loss,
            'Test': avg_test_loss
        },
        epoch + 1
    )

```

```
writer.add_scalar(  
    'Test Accuracy',  
    test_accuracy,  
    epoch + 1  
)  
  
writer.flush()  
  
torch.save(model.state_dict(), 'spiral_model.pth')
```

EPOCH 100:
LOSS train 0.3527 test 0.3550
Test accuracy: 0.9097 (272 / 299 test samples).

EPOCH 200:
LOSS train 0.1385 test 0.1413
Test accuracy: 0.9498 (284 / 299 test samples).

EPOCH 300:
LOSS train 0.0736 test 0.0856
Test accuracy: 0.9732 (291 / 299 test samples).

EPOCH 400:
LOSS train 0.0484 test 0.0640
Test accuracy: 0.9799 (293 / 299 test samples).

EPOCH 500:
LOSS train 0.0353 test 0.0484
Test accuracy: 0.9900 (296 / 299 test samples).

EPOCH 600:
LOSS train 0.0268 test 0.0388
Test accuracy: 0.9967 (298 / 299 test samples).

EPOCH 700:
LOSS train 0.0220 test 0.0318
Test accuracy: 0.9933 (297 / 299 test samples).

EPOCH 800:
LOSS train 0.0179 test 0.0262
Test accuracy: 0.9933 (297 / 299 test samples).

EPOCH 900:
LOSS train 0.0153 test 0.0234
Test accuracy: 0.9967 (298 / 299 test samples).

EPOCH 1000:
LOSS train 0.0132 test 0.0221

Test accuracy: 0.9967 (298 / 299 test samples).

3.2.3 Describe your results and discuss the observed performance

After training the model for 1000 epochs it correctly classified 298/299 samples in the test set, achieving an accuracy of 99.67%. The training loss on the 1000th epoch was 0.0132 while the test loss was a bit higher at 0.0221. This shows that the model did not overfit to the training data and could generalize on unseen data although with slightly worse performance, which is to be expected. Only missclassifying 1 sample indicates that the model has successfully learned a nonlinear decision boundary that at least approximates the true decision boundary of the data.

3.2.4 Visualize network performance similar to:

```
model= #trained pytorch network
import numpy as np
import matplotlib.pyplot as plt

model.cpu()
x,y=np.meshgrid(np.linspace(-1,1,30),np.linspace(-1,1,30))
xy=np.concatenate((x.reshape(-1,1),y.reshape(-1,1)),axis=1)
z=model(torch.tensor(xy).float()).detach().numpy()
z=np.argmax(z,1).reshape(30,30)
plt.contourf(x,y,z)
plt.scatter(testX[:,0],testX[:,1],c=testLabels)
```

```
[13]: model = SpiralModel()
model.load_state_dict(torch.load('spiral_model.pth'))
model.cpu()
x,y=np.meshgrid(np.linspace(-1,1,30),np.linspace(-1,1,30))
xy=np.concatenate((x.reshape(-1,1),y.reshape(-1,1)),axis=1)
z=model(torch.tensor(xy).float()).detach().numpy()
z=np.argmax(z,1).reshape(30,30)
plt.contourf(x,y,z)
plt.scatter(test_data["x"],test_data["y"],c=test_data["label"])
```

```
[13]: <matplotlib.collections.PathCollection at 0x28fb1b06d50>
```

